Chapter 4 — Interprocess Communication (IPC) & Synchronization

> **In one sentence:** IPC is how separate processes talk to each other and take turns so they don't corrupt shared data.

---

1. The big picture (a simple analogy)

Imagine two chefs (processes) working in one kitchen (the computer). They share the fridge and the stove (shared memory / resources).

- They need to **talk** to each other → *communication*.
- They must not both grab the same pot at the same moment → *synchronization*.

If they don't coordinate, dinner (the program output) gets ruined. That is exactly why operating systems provide **IPC** and **synchronization** tools.

---

2. Key definitions

Term	Meaning
Process	A program in execution — a series of actions.
IPC	A mechanism that lets processes communicate with each other and synchronize their actions.
Independent process	Cannot be affected by other processes running in the system.
Cooperating process	Can be affected by other processes — i.e. it **shares data** with another process.

> ⚠ **Any process that shares data with another process is a cooperating process.**

---

3. Why processes cooperate (and why we need IPC)

Processes cooperate for these reasons:

Reason	Explanation
Information sharing	Processes need to share tasks/data (files, printer, scanner).
Resource sharing	Hardware devices and files must be shared.
Computation speedup	Split work into parallel parts.

Reason	Explanation
Synchronization	Keep actions in the right order.
Modularity	Divide a big task into small modules.
Convenience	Easier to build systems from cooperating parts.

Why IPC is needed:

- Each process runs sequentially on its own.
- Problems appear only when processes **share data**:
  - exchange data between processes,
  - let processes enter **critical regions** safely,
  - keep actions in **proper sequence**.
- Threads have the same issues (they share memory easily; ordering and critical-region issues still apply).

---

4. Two fundamental IPC models

4.1 Shared memory

- The OS sets up a **region of memory** that cooperating processes can both access.
- Processes exchange information by **reading and writing** to that region.

Analogy: two people writing on the same whiteboard.

✓ Very fast (no copying through the OS after setup). ✗ Synchronization is the programmer's job — easy to create race conditions.

4.2 Message passing

- Processes communicate by **sending and receiving messages** — no shared address space needed.
- Great for **distributed systems** where processes live on different computers connected by a network.
- Basic primitives: `send()` and `receive()`.

Analogy: sending letters or texts instead of sharing a whiteboard.

✓ Works across machines, clean interface. ✗ Slower (messages are copied through the OS/kernel).

Comparison

Feature	Shared memory	Message passing
Address space	Shared region	Separate
Speed	Fast	Slower
Best for	Same machine, heavy data	Distributed systems
Synchronization	Programmer's responsibility	Built into send/receive semantics

Feature	Shared memory	Message passing
Example	`shmget`, `mmap`	sockets, pipes, MPI

---

5. Message passing — the details

5.1 How two processes set up communication

1. **Establish a communication link.**
2. **Exchange messages** using `send()` and `receive()`.

5.2 Naming — how do processes address each other?

Style	How it works	Example
Direct communication	Name the other process explicitly	`send(Q, message)`, `receive(P, message)`
Indirect communication	Send to a **mailbox / port**, receiver reads from it	P → mailbox/port → Q

5.3 Synchronization — blocking or not?

Style	Behaviour
Synchronous (blocking)	Sender/receiver **blocks** until the message is sent/received.
Asynchronous (non-blocking)	Sender/receiver **continues immediately**; message is buffered.

5.4 Buffering — how many messages can wait?

Buffer	Meaning
Zero capacity	No queue — sender must wait for receiver (rendezvous).
Finite capacity	Bounded queue; sender blocks when full.
Infinite capacity	Sender never blocks (theoretically).

---

6. Synchronization — the core idea

> **Synchronization** = coordinating the activities of two or more processes so they can: 1. **compete for resources in a mutually exclusive way**, and 2. **cooperate in sequencing events** (do things in the right order).

6.1 The problem: race conditions

- Cooperating processes share storage (memory).
- Both may **read and write** the same data.
- The OS cannot guarantee that "read then write" happens as one indivisible (**atomic**) step.
- **Order matters** → wrong interleaving gives **wrong results**.

Classic example — `counter++`

```
counter++;   // actually: load counter → add 1 → store counter
```

If two processes both do `counter++` at the same time:

Time	Process A	Process B	counter
t1	read counter (= 5)		5
t2		read counter (= 5)	5
t3	add 1 → 6		5
t4		add 1 → 6	5
t5	write 6		6
t6		write 6	6

Expected: `7`. Got: `6`. **That is a race condition.**

---

7. Critical regions (critical sections)

> A **critical region** is a piece of code that accesses shared data and must not be executed by more than one process at a time.

Use critical regions to provide **mutual exclusion** and eliminate race conditions.

7.1 The four conditions for a correct solution

1. **Mutual exclusion** — no two processes may be inside their critical regions at the same time.
2. **No assumptions** — nothing is assumed about speeds or the number of CPUs.
3. **No unnecessary blocking** — a process outside its critical region must not block another process from entering.
4. **Bounded waiting** — no process should wait forever to enter its critical region (no starvation).

7.2 Busy waiting / spin lock (the `turn` variable)

```
// Process 0                    // Process 1
while (TRUE) {                  while (TRUE) {
    while (turn != 0) ;             while (turn != 1) ;
    critical_region();              critical_region();
    turn = 1;                       turn = 0;
    noncritical_region();           noncritical_region();
}                               }
```

- A shared variable `turn` says whose turn it is.
- The waiting process keeps reading it in a tight loop → the CPU is **wasting time spinning**.
- This is called a **spin lock**.
- ✓ It prevents race conditions.

- ✗ It **fails condition 3**: if Process 0 is outside its critical region, Process 1 may still be blocked forever waiting for `turn`.

---

8. Semaphores

A **semaphore** is an integer variable accessed only through two atomic operations:

Operation	Meaning
`wait(S)` (a.k.a. P / down)	Wait until `S > 0`, then `S = S − 1`.
`signal(S)` (a.k.a. V / up)	`S = S + 1`, waking a waiting process.

Types

- **Binary semaphore** — values 0 or 1 → acts like a **lock / mutex**.
- **Counting semaphore** — any non-negative value → controls access to a pool of N resources.

Analogy: a semaphore is like a set of keys to a room. `wait` takes a key; `signal` returns one.

---

9. The bounded-buffer problem (producer–consumer)

Setup

- A buffer has `N` slots.
- **Producer** puts items in; **consumer** takes items out.
- Rules:
  - Producer must **not insert when full**.
  - Consumer must **not remove when empty**.
  - They must **not access the buffer at the same time**.

Bad version (busy waiting)

```
// Producer                          // Consumer
while (Items_number == buffer_size)  while (Items_number == 0)
    ; // wait: buffer full                 ; // wait: buffer empty
Buffer[i] = next_produced_item;      Consumed_item = buffer[j];
i = (i + 1) % Buffer_size;           j = (j + 1) % Buffer_size;
Items_number++;                      Items_number--;
```

This busy-waits and still has race conditions on `Items_number`.

Correct version — using three semaphores

Semaphore	Type	Initial value	Purpose
`mutex`	binary	1	Lock the buffer (mutual exclusion)
`empty`	counting	`N` (buffer size)	Number of free slots
`full`	counting	0	Number of filled slots

Producer

```
do {
    wait(empty);        // wait for a free slot (empty > 0), then empty--
    wait(mutex);        // acquire the lock
    /* insert item into buffer */
    signal(mutex);      // release the lock
    signal(full);       // full++
} while (TRUE);
```

Consumer

```
do {
    wait(full);         // wait for an item (full > 0), then full--
    wait(mutex);        // acquire the lock
    /* remove item from buffer */
    signal(mutex);      // release the lock
    signal(empty);      // empty++
} while (TRUE);
```

Why the order matters: always `wait` on the resource counter (`empty` / `full`) **before** `wait(mutex)`. If you lock first and then find the buffer full, you would sleep while holding the lock → **deadlock**.

---

10. Monitors

A **monitor** is a high-level synchronization construct (usually built into a language).

- Only **one process at a time** can be active inside the monitor.
- All data inside is hidden and accessed only through the monitor's procedures.
- Mutual exclusion is **automatic** → fewer programmer mistakes.

```
monitor mon {
    int foo;
    int bar;
    double arr[100];

    void proc1(…) { }
    void proc2(…) { }

    void mon() { /* initialization */ }
};
```

- `foo`, `bar`, `arr` are visible only to `proc1` and `proc2`.
- Only one process may run inside `proc1` or `proc2` at any moment.
- Looks like C++ but **is not** supported by C++; monitors need language support.
- Can be implemented with semaphores.

10.1 Condition variables (waiting inside a monitor)

Problem: how can a process wait inside a monitor? It can't simply sleep — then nobody could enter to wake it.

Solution: a **condition variable** with two operations:

Operation	Effect
`Wait()`	Suspend this process until it is signaled. **Releases the lock** while waiting.
`Signal()`	Wake exactly **one** waiting process.

Rules: - If no process is waiting, `signal` does **nothing** (signals are **not saved up**). - Condition variables are usable **only inside the monitor**; you must hold the lock.

10.2 Mesa vs Hoare semantics

When P signals and wakes Q, both want the monitor. Who runs first?

Semantics	Who continues first?	When does the other resume?
Mesa	P (the signaler)	Q resumes when P leaves the monitor
Hoare	Q (the awakened process)	P resumes when Q leaves the monitor

Hoare can be safer because the condition Q was waiting for might no longer be true if P keeps running.

10.3 Locks + condition variables (manual monitors)

Concept	Usage
`Acquire()`	Entering the monitor
`Release()`	Leaving the monitor
`Wait()`	Waiting on a condition (implicitly releases the lock)
`Signal()`	Wake one waiter

Each condition variable belongs to exactly **one** lock; the lock must be held to use it; returning from `Wait()` re-acquires the lock.

---

11. Barriers

- Used to synchronize **many** processes.
- Each process waits at the **barrier** until **all** processes in the group arrive.
- Once all have arrived, **all proceed together**.
- Can be implemented with locks + condition variables.

Analogy: a group of hikers waiting at a checkpoint until everyone catches up.

```
Approaching barrier →  B and D waiting →  All arrived →  Everyone released
```

Useful for parallel algorithms where every phase must finish before the next begins.

12. Deadlock and starvation

Problem	Definition	Example
Deadlock	Two or more processes wait **forever** for an event that can only be caused by another waiting process.	P0 holds A, needs B; P1 holds B, needs A.
Starvation	A process is **indefinitely blocked** — it never gets the resource (e.g. never removed from a semaphore queue).	Priority ordering always favours other processes.

Difference: in deadlock the whole set is stuck; in starvation one process is forgotten while others keep making progress.

13. Dining Philosophers problem

Story

- `N` philosophers sit around a table.
- Between each pair is **one chopstick** (total `N`).
- A philosopher needs **two** chopsticks to eat.
- Philosophers alternate between **thinking** and **eating**.
- **Goal:** coordinate chopstick use so everyone eats and no one gets stuck.

Naive solution: each philosopher picks up the left chopstick, then the right. → If everyone picks up the left chopstick at once, **everyone waits forever** for the right one → **deadlock**.

Better solution (solution 2 in the slides):

- One semaphore per chopstick.
- A hungry philosopher picks up the **lower-numbered** chopstick first, then the **higher-numbered** one, eats, then puts both down.

Still to consider: - **Deadlock** — avoided by the ordering rule. - **Fairness / starvation** — some philosopher may still wait a long time.

14. Quick revision table

Concept	One-line summary
IPC	Mechanism for processes to communicate + synchronize
Independent vs cooperating	Independents don't affect others; cooperators share data
Shared memory	Fast; shared region; manual synchronization
Message passing	send/receive; good for distributed systems
Race condition	Result depends on timing of shared access
Critical region	Code that must run by only one process at a time
Mutual exclusion	Guarantee only one process is in the critical region
Spin lock	Busy-wait loop on a variable (`turn`)

Concept	One-line summary
Semaphore	Counter with atomic `wait()` / `signal()`
Mutex	Binary semaphore used as a lock
Monitor	Language construct allowing only one process inside
Condition variable	`Wait()` / `Signal()` to sleep/wake inside a monitor
Mesa vs Hoare	Signaler runs first vs awakened runs first
Barrier	Everyone waits until all arrive
Deadlock	Circular waiting, nobody progresses
Starvation	One process is forever denied
Dining philosophers	Classic deadlock/starvation illustration

15. Formulas / code to memorise

```
Semaphore wait(S):   while (S <= 0) ;   S--;
Semaphore signal(S): S++;
```

```
Bounded buffer:
  mutex = 1, empty = N, full = 0

  Producer: wait(empty); wait(mutex);  ...  signal(mutex); signal(full);
  Consumer: wait(full);  wait(mutex);  ...  signal(mutex); signal(empty);
```

16. Likely exam questions (and answer hints)

1. **Define IPC and explain why it is needed.** → Definition + cooperating processes + data sharing, sequencing, critical regions.
2. **Compare shared memory and message passing.** → Table in §4; mention speed, address space, distributed systems.
3. **What is a race condition? Give an example.** → `counter++` interleaving table in §6.1.
4. **State the four conditions for a correct critical-region solution.** → §7.1 (mutual exclusion, no assumptions, no unnecessary blocking, bounded waiting).
5. **Solve the bounded-buffer problem using semaphores.** → §9 code + explain the three semaphores and the order of `wait` calls.
6. **What is a monitor? How does it differ from a semaphore?** → Monitor = high-level, automatic mutual exclusion, condition variables; semaphore = low-level counter, manual.
7. **Explain deadlock vs starvation.** → §12 table + dining philosophers example.
8. **Explain Mesa and Hoare monitor semantics.** → §10.2 table.

17. Common mistakes to avoid

- ✗ Writing `wait(mutex)` **before** `wait(empty)` / `wait(full)` in the producer/consumer → **deadlock**.
- ✗ Forgetting that `wait()` decrements and `signal()` increments.
- ✗ Thinking a spin lock is efficient — it **burns CPU** while waiting.
- ✗ Confusing **deadlock** (circular wait) with **starvation** (one process ignored).
- ✗ Saying signals on condition variables are remembered — they are **not saved up**.
- ✗ Assuming monitors need no synchronization — they give it automatically, but only inside the monitor.